

Software Engineering II – Coding Standards Lab 2026

Escuela Superior Politécnica del Litoral – Faculty of Electrical and Computer Engineering

Alfredo Andrés Porras Mendez

Objectives

- Use a code quality tool to detect and enforce coding standards.
- Refactor and complete a faulty base program so it meets a given set of functional requirements.
- Obtain HTML reports of the issues detected in the code, before and after your changes.

Requirements

- Visual Studio Code
- Python (Section A)
- Eclipse IDE for Java Developers (optional, Section B)
- Git and a GitHub or GitLab account

Introduction

Code conventions and best practices keep a codebase consistent and readable. Following them improves code quality, eases collaboration and reduces errors. Enforcing them by hand is tedious, so we use automated tools.

Linting tools analyze source code and flag deviations from coding standards. They check class design, method design, layout and formatting, and can produce project-wide reports. Typical check categories include:

- Annotations, block checks, class design, coding
- Headers, imports, Javadoc comments, metrics
- Modifiers, naming conventions, size violations, whitespace

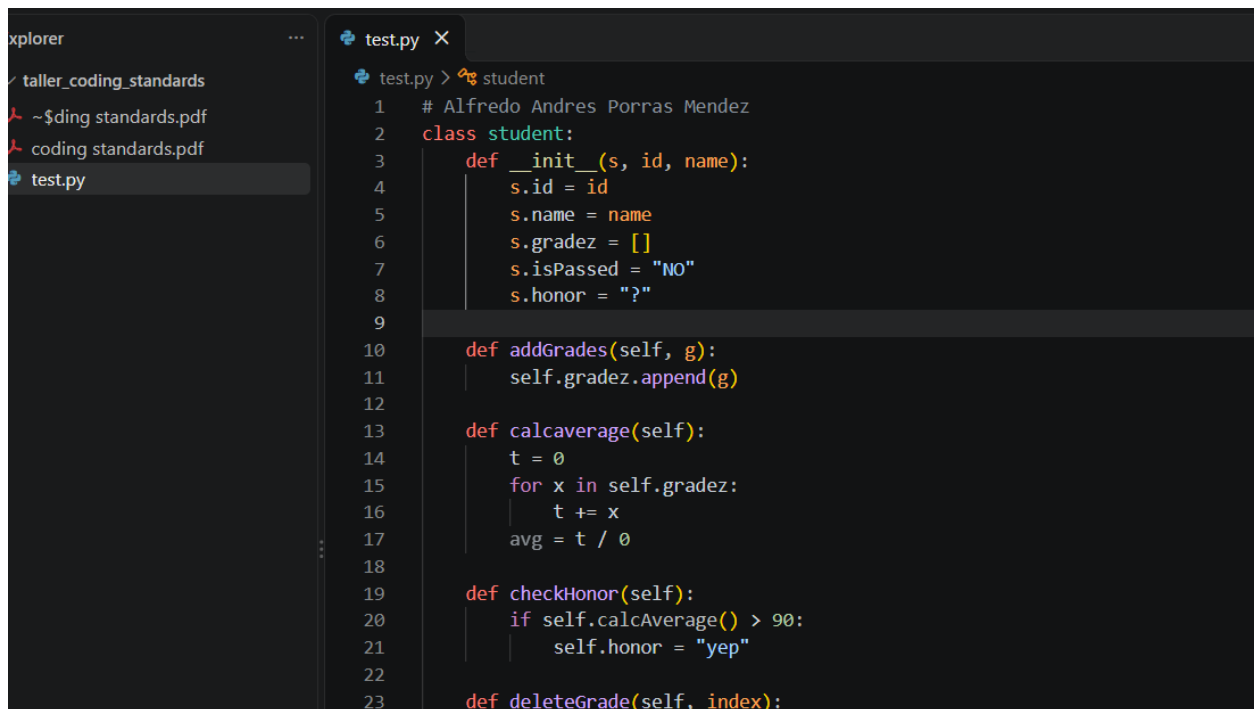
In this lab you receive a base program that is intentionally poorly written and full of errors. Your job is to use a coding standards tool to find the problems, then rewrite the program so it follows good practices and meets all the functional requirements listed in the Functional Requirements section. The lab offers two versions, one per language: Section A (Python).

Python will be used for this workshop because of familiarity, experience and efficiency.

Development

Step 1 – Choose the programming language

1. I chose Python because of experience and familiarity with this language.

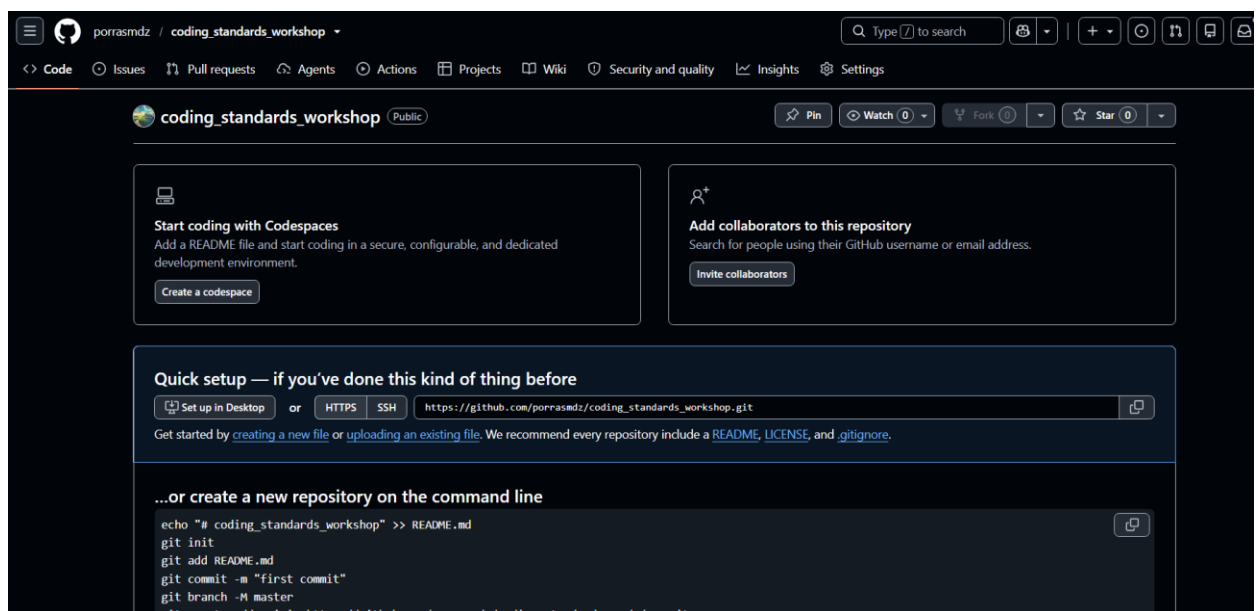


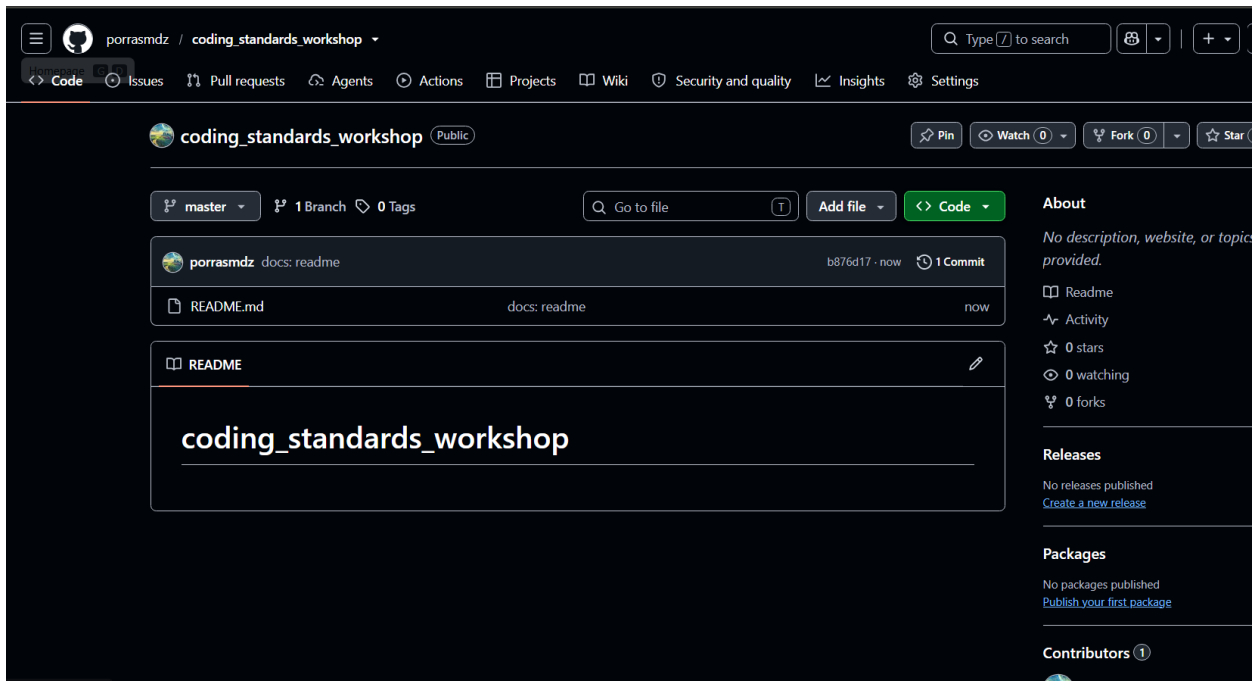
```
test.py > student
1 # Alfredo Andres Porras Mendez
2 class student:
3     def __init__(s, id, name):
4         s.id = id
5         s.name = name
6         s.grades = []
7         s.isPassed = "NO"
8         s.honor = "?"
9
10    def addGrades(self, g):
11        self.grades.append(g)
12
13    def calcaverage(self):
14        t = 0
15        for x in self.grades:
16            t += x
17        avg = t / 0
18
19    def checkHonor(self):
20        if self.calcAverage() > 90:
21            self.honor = "yep"
22
23    def deleteGrade(self, index):
```

Step 2 – Create a repository with the base code

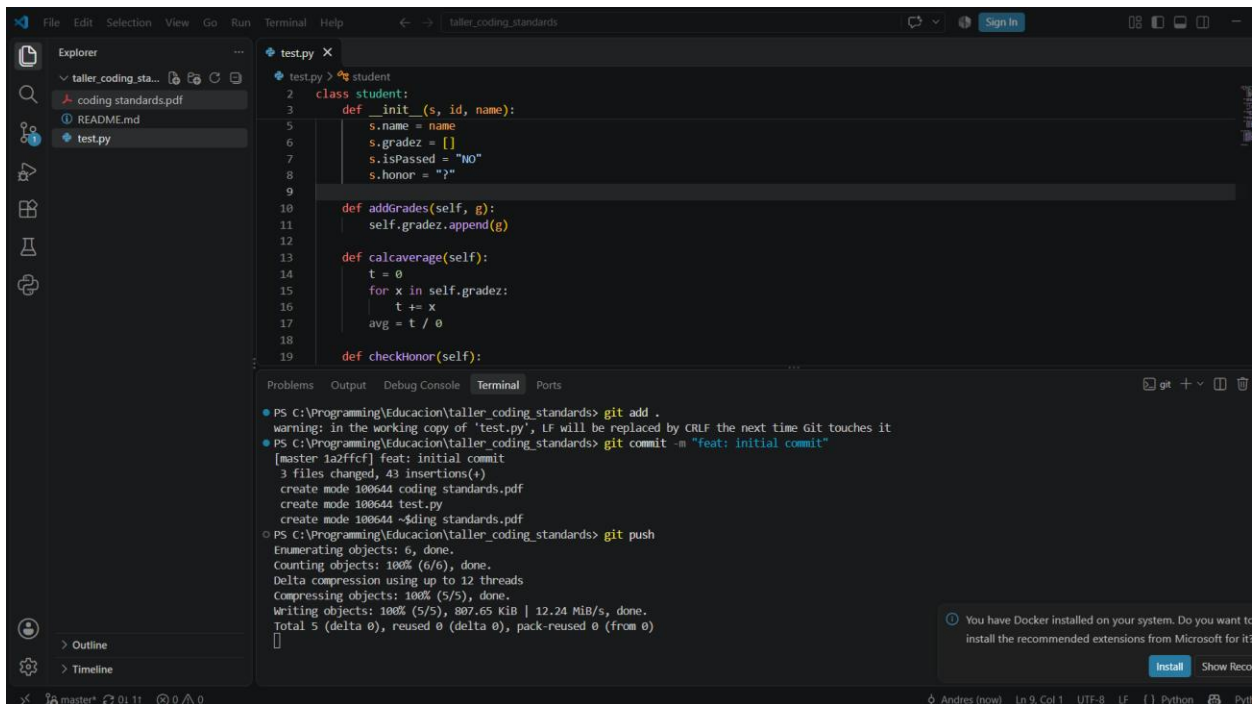
1. Create a new public repository on GitHub or GitLab for your chosen language. Give it a meaningful name and initialize it with a README file.

https://github.com/porrasmdz/coding_standards_workshop





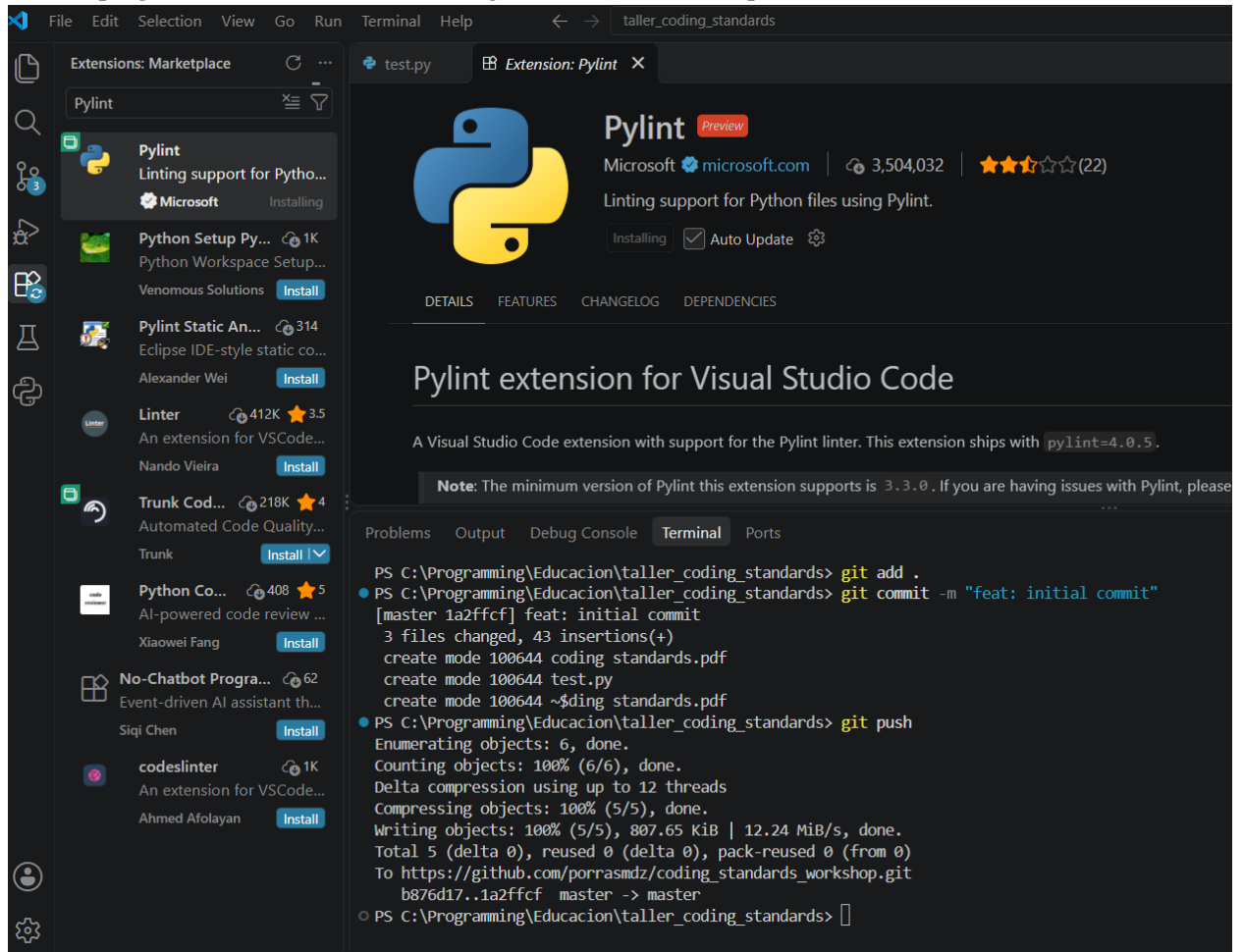
2. Clone the repository and open it in Visual Studio Code.
3. Create the source file(s) and copy the base code of your section exactly as given. Commit and push this first version.

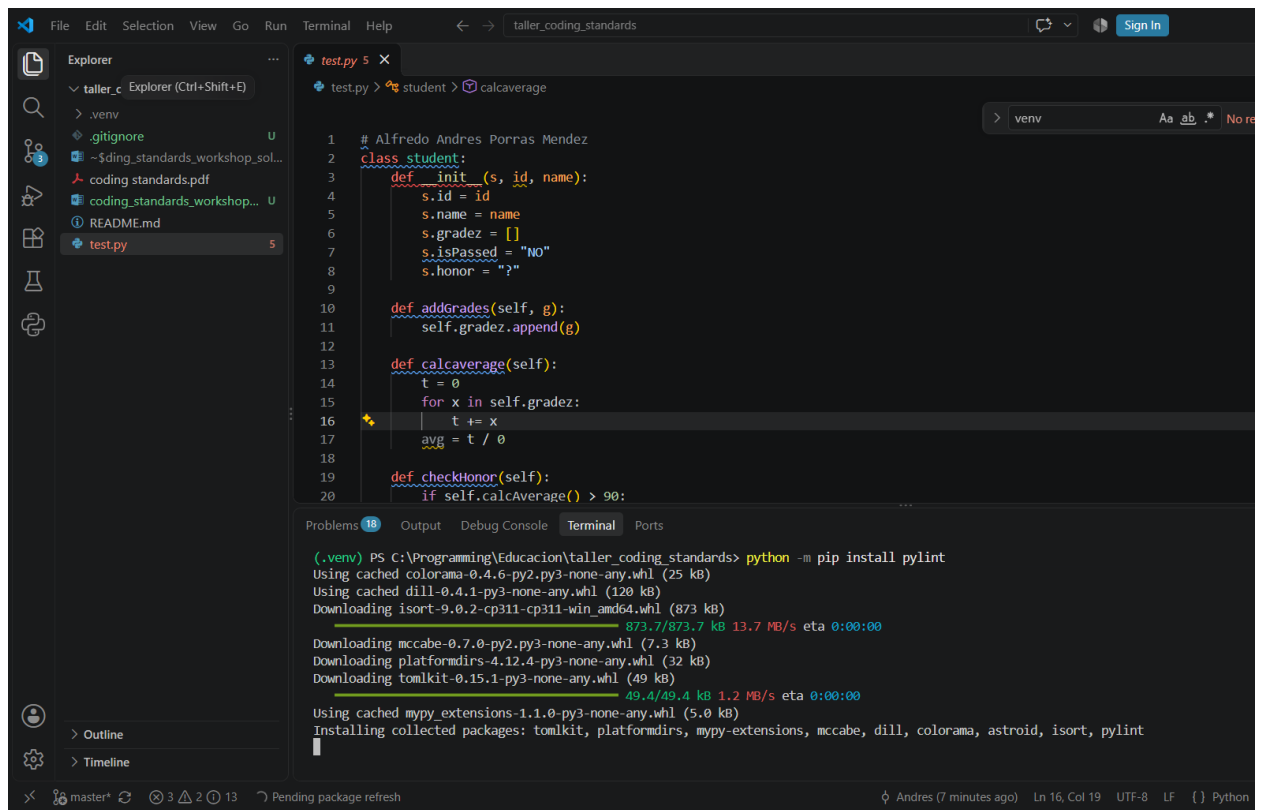


Challenge (5 pts): Create a GitHub Actions workflow that runs your coding standards tool on every pull request to the main branch.

Step 3 – Run the code standards tool on the base code (initial report)

1. Research automated code standards tools for your language that work with Visual Studio Code. I will be using Pylint for this workshop
2. Install the tool, either as a VS Code extension or directly (e.g. `pip install pylint`). I will be using both the plugin and the cli tool in order to generate corrections report and have a visual feedback.





3. Run the analysis on the unchanged base code.

```
pylint_base_report.txt
1 ***** Module test
2 test.py:1:0: C0114: Missing module docstring (missing-module-docstring)
3 test.py:2:0: C0115: Missing class docstring (missing-class-docstring)
4 test.py:2:0: C0103: Class name "student" doesn't conform to PascalCase naming style (invalid-name)
5 test.py:7:8: C0103: Attribute name "isPassed" doesn't conform to snake_case naming style (invalid-name)
6 test.py:3:4: E0213: Method '__init__' should have "self" as first argument (no-self-argument)
7 test.py:3:20: W0622: Redefining built-in 'id' (redefined-builtin)
8 test.py:10:4: C0116: Missing function or method docstring (missing-function-docstring)
9 test.py:10:4: C0103: Method name "addGrades" doesn't conform to snake_case naming style (invalid-name)

Problems Output Debug Console Terminal Ports

(.venv) PS C:\Programming\Educacion\taller_coding_standards> python -m pip install pylint
(.venv) PS C:\Programming\Educacion\taller_coding_standards> pylint .\test.py
***** Module test
test.py:1:0: C0114: Missing module docstring (missing-module-docstring)
test.py:2:0: C0115: Missing class docstring (missing-class-docstring)
test.py:2:0: C0103: Class name "student" doesn't conform to PascalCase naming style (invalid-name)
test.py:7:8: C0103: Attribute name "isPassed" doesn't conform to snake_case naming style (invalid-name)
test.py:3:4: E0213: Method '__init__' should have "self" as first argument (no-self-argument)
test.py:3:20: W0622: Redefining built-in 'id' (redefined-builtin)
test.py:10:4: C0116: Missing function or method docstring (missing-function-docstring)
test.py:10:4: C0103: Method name "addGrades" doesn't conform to snake_case naming style (invalid-name)
test.py:13:4: C0116: Missing function or method docstring (missing-function-docstring)
test.py:17:8: W0612: Unused variable 'avg' (unused-variable)
test.py:19:4: C0116: Missing function or method docstring (missing-function-docstring)
test.py:19:4: C0103: Method name "checkHonor" doesn't conform to snake_case naming style (invalid-name)
test.py:20:11: E1101: Instance of 'student' has no 'calcAverage' member; maybe 'calccaverage'? (no-member)
test.py:23:4: C0116: Missing function or method docstring (missing-function-docstring)
test.py:23:4: C0103: Method name "deleteGrade" doesn't conform to snake_case naming style (invalid-name)
test.py:26:4: C0116: Missing function or method docstring (missing-function-docstring)
test.py:30:33: E1101: Instance of 'student' has no 'letter' member (no-member)
test.py:33:0: C0116: Missing function or method docstring (missing-function-docstring)

-----
Your code has been rated at 0.91/10

(.venv) PS C:\Programming\Educacion\taller_coding_standards> pylint .\test.py > ./pylint_base_report.txt
```

4. Save the generated report (TXT in this case) and take screenshots of the errors and violations found. This is your initial report.

```
pylint_base_report.txt
> venv Aa ab

1 ***** Module test
2 test.py:1:0: C0114: Missing module docstring (missing-module-docstring)
3 test.py:2:0: C0115: Missing class docstring (missing-class-docstring)
4 test.py:2:0: C0103: Class name "student" doesn't conform to PascalCase naming style (invalid-name)
5 test.py:7:8: C0103: Attribute name "isPassed" doesn't conform to snake_case naming style (invalid-name)
6 test.py:3:4: E0213: Method '__init__' should have "self" as first argument (no-self-argument)
7 test.py:3:20: W0622: Redefining built-in 'id' (redefined-builtin)
8 test.py:10:4: C0116: Missing function or method docstring (missing-function-docstring)
9 test.py:10:4: C0103: Method name "addGrades" doesn't conform to snake_case naming style (invalid-name)
10 test.py:13:4: C0116: Missing function or method docstring (missing-function-docstring)
11 test.py:17:8: W0612: Unused variable 'avg' (unused-variable)
12 test.py:19:4: C0116: Missing function or method docstring (missing-function-docstring)
13 test.py:19:4: C0103: Method name "checkHonor" doesn't conform to snake_case naming style (invalid-name)
14 test.py:20:11: E1101: Instance of 'student' has no 'calcAverage' member; maybe 'calcaverage'? (no-member)
15 test.py:23:4: C0116: Missing function or method docstring (missing-function-docstring)
16 test.py:23:4: C0103: Method name "deleteGrade" doesn't conform to snake_case naming style (invalid-name)
17 test.py:26:4: C0116: Missing function or method docstring (missing-function-docstring)
18 test.py:30:33: E1101: Instance of 'student' has no 'letter' member (no-member)
19 test.py:33:0: C0116: Missing function or method docstring (missing-function-docstring)
20
21 -----
22 Your code has been rated at 0.91/10 (previous run: 0.91/10, +0.00)
23
24
```

Step 4 – Refactor and complete the code (iterative process)

The base code does not work and does not meet the requirements. Your task is to turn it into a working program that:

- Fixes all the errors and violations reported by the tool;

```
pylint_base_report.txt U test.py M X
test.py > test.py
3 class Student:
25     def check_honor(self):
26         """Checks if student has honourable grade and assign it"""
27         if self.calc_average() > 90:
28             self.honor = "yep"
29
30     def get_letter(self):
31         """Gets letter for average grade"""
32         avg = self.calc_average()
33         letter = "F"
34
35         if avg >= 90:
36             letter = "A"
37         elif avg >= 80:
38             letter = "B"
39         elif avg >= 70:
40             letter = "C"
41         elif avg >= 60:
42             letter = "D"
43         else:
44             letter = "F"
45
46     def calc_average(self):
47         """Calculates the average grade"""
48         avg = 0
49         for grade in self.grades:
50             avg += grade
51         return avg / len(self.grades)
52
53     def __str__(self):
54         return f"Student {self.name} with average grade {self.calc_average():.2f} and honor {self.honor}"
55
56     def __repr__(self):
57         return f"Student({self.name}, {self.grades}, {self.honor})"
58
59 if __name__ == '__main__':
60     student = Student("John", [85, 92, 78, 88, 95])
61     student.calc_average()
62     student.get_letter()
63     student.check_honor()
64     print(student)
65     print(student.__repr__())
```

Problems Output Debug Console Terminal Ports

```
(.venv) PS C:\Programming\Educacion\taller_coding_standards> pylint .\test.py
-----
Your code has been rated at 9.79/10 (previous run: 8.53/10, +1.26)

(.venv) PS C:\Programming\Educacion\taller_coding_standards> pylint .\test.py
*****
Module test
test.py:29:0: C0303: Trailing whitespace (trailing-whitespace)
-----
Your code has been rated at 9.79/10 (previous run: 9.79/10, +0.00)

(.venv) PS C:\Programming\Educacion\taller_coding_standards> pylint .\test.py
-----
Your code has been rated at 10.00/10 (previous run: 9.79/10, +0.21)

(.venv) PS C:\Programming\Educacion\taller_coding_standards>
```

- follows good practices the tool may not catch (e.g. meaningful names, no magic values, proper error handling);
Meaningful names assigned, error validations, no magic values, proper error handling, main function rename.

```

:...skipping...
warning: in the working copy of 'test.py', LF will be replaced by CRLF the next time Git touches it
diff --git a/test.py b/test.py
index 1f39aec..7274426 100644
--- a/test.py
+++ b/test.py
@@ -1,71 +1,101 @@
 """Module providin the solution for Coding Standards Workshop"""

+from enum import IntEnum
+
+class GradeThreshold(IntEnum):
+    """Minimum average required for each letter grade."""
+    A = 90
+    B = 80
+    C = 70
+    D = 60
+
+class Student:
+    """Class representing a student and its utilities in this workshop program"""
+    def __init__(self, user_id, name):
+        self.id = user_id
+        self.name = name
+        self.gradez = []
+        self.is_passed = "NO"
+        self.honor = "?"
+        self.grades = []
+        self.is_passed = False
+        self.honor = False
+
+    def add_grade(self, grade):
+        """Add a valid grade to the student."""
+        if isinstance(grade, bool) or not isinstance(grade, (int, float)):
+            raise TypeError("Grade must be numeric.")
+
+    def add_grades(self, g):
+        """Add a grade to this student instance"""
+        self.gradez.append(g)

```



```

        return letter

def delete_grade(self, index):
    """Removes a grade in student grades"""
    del self.grades[index]

def report(self): # broken format
    """Determine the letter grade based on the average."""
    average = self.calc_average()

    if average >= GradeThreshold.A:
        return "A"
    if average >= GradeThreshold.B:
        return "B"
    if average >= GradeThreshold.C:
        return "C"
    if average >= GradeThreshold.D:
        return "D"

    return "F"

def delete_grade(self, index: int):
    """Remove a grade by its index."""

    if index < 0 or index >= len(self.grades):
        raise IndexError("Grade index is out of range.")

    del self.grades[index]

def report(self):
    """Generates user report"""
    print("ID: " + self.id)
    print("Name is: " + self.name)
    print("Grades Count: " + len(self.grades))
    print("Final Grade = " + self.get_letter())
    print(f"ID: {self.id}")
    print(f"Name: {self.name}")
    print(f"Grades Count: {len(self.grades)}")

```

```

+
+if __name__ == "__main__":
+    student = Student("x", "")
+
+    try:
+        student.add_grade(100)
+        student.add_grade("Fifty") # TypeError
+    except (TypeError, ValueError) as error:
+        print(f"Error adding grade: {error}")
+
-def startrun():
-    """Runs the current program"""
-    a = Student("x", "")
-    a.add_grades(100)
-    a.add_grades("Fifty") # broken
-    a.calc_average()
-    a.check_honor()
-    a.delete_grade(5) # IndexError
-    a.report()
+    try:
+        student.calc_average()
+        student.check_honor()
+    except ValueError as error:
+        print(f"Error calculating grades: {error}")
+
+    try:
+        student.delete_grade(5) # IndexError
+    except (IndexError, TypeError) as error:
+        print(f"Error deleting grade: {error}")
+
-def startrun()
+    try:
+        student.report()
+    except (ValueError, TypeError) as error:
+        print(f"Error generating report: {error}")
+
+ (END)

```

Error handling

```
test.py > ...
82     student.add_grade(100)
83     student.add_grade("Fifty") # TypeError
84 except (TypeError, ValueError) as error:
85     print(f"Error adding grade: {error}")
86
87     try:
88         student.calc_average()
89         student.check_honor()
90     except ValueError as error:
91         print(f"Error calculating grades: {error}")
92
93     try:
94         student.delete_grade(5) # IndexError
95     except (IndexError, TypeError) as error:
96         print(f"Error deleting grade: {error}")
97
98     try:
99         print("#####")
100         student.report()
101     except (ValueError, TypeError) as error:
```

Problems Output Debug Console Terminal Ports

```
(.venv) PS C:\Programming\Educacion\taller_coding_standards> python .\test.py
ID: x
Name:
Grades Count: 1
Final Grade: A
(.venv) PS C:\Programming\Educacion\taller_coding_standards> python .\test.py
Error adding grade: Grade must be numeric.
Error deleting grade: Grade index is out of range.
#####
ID: x
Name:
Grades Count: 1
Final Grade: A
(.venv) PS C:\Programming\Educacion\taller_coding_standards> 
```

Error Handling Id and Name

```
test.py > Student > check_honor
11
12 class Student:
13     """Class representing a student and its utilities in this workshop program"""
14     def __init__(self, user_id, name):
15         if not isinstance(user_id, str) or not user_id.strip():
16             raise ValueError("Student ID cannot be empty.")
17
18         if not isinstance(name, str) or not name.strip():
19             raise ValueError("Student name cannot be empty.")
20
21         self.id = user_id
22         self.name = name
23         self.grades = []
24         self.is_passed = False
25         self.honor = False
26
27     def add_grade(self, grade):
28         """Add a valid grade to the student."""
29         if isinstance(grade, bool) or not isinstance(grade, (int, float)):
30             raise TypeError("Grade must be numeric.")
31
Problems 2 Output Debug Console Terminal Ports
(.venv) PS C:\Programming\Educacion\taller_coding_standards> python .\test.py
Final Grade: A
(.venv) PS C:\Programming\Educacion\taller_coding_standards> python .\test.py
#####
ERROR HANDLING EXAMPLE
#####
Traceback (most recent call last):
  File "C:\Programming\Educacion\taller_coding_standards\test.py", line 89, in <module>
    student = Student("x", "")
    ^^^^^^^^^^^^^^^^^^^^^
  File "C:\Programming\Educacion\taller_coding_standards\test.py", line 19, in __init__
    raise ValueError("Student name cannot be empty.")
ValueError: Student name cannot be empty.
(.venv) PS C:\Programming\Educacion\taller_coding_standards> []
```

- implements all the requirements in the Functional Requirements section, including the extended ones.

```
(.venv) PS C:\Programming\Educacion\taller_coding_standards> python .\test.py
```

```
===== 1. ADD STUDENTS =====
```

```
Student created: 001 - John Doe
```

```
===== 2. ADD GRADES =====
```

```
Grade added: 95.0
```

```
Grade added: 72.5
```

```
Grade added: 88.0
```

```
Current grades: [95.0, 72.5, 88.0]
```

```
===== 3. CALCULATE AVERAGE =====
```

```
Average grade: 85.17
```

```
===== 4. DETERMINE LETTER GRADE =====
```

```
Average: 95 -> Letter: A
```

```
Average: 85 -> Letter: B
```

```
Average: 75 -> Letter: C
```

```
Average: 65 -> Letter: D
```

```
Average: 55 -> Letter: F
```

```
===== 5. DETERMINE PASS/FAIL =====
```

```
Average: 90 -> Passed
```

```
Average: 60 -> Passed
```

```
Average: 59 -> Failed
```

```
===== 6. HANDLE INVALID INPUTS =====
```

```
Invalid student: Student ID cannot be empty.
```

```
Invalid student: Student name cannot be empty.
```

```
Invalid grade (Fifty): Grade must be numeric.
```

```
Invalid grade (-10): Grade must be between 0 and 100.
```

```
Invalid grade (150): Grade must be between 0 and 100.
```

```
===== 7. HONOR ROLL DETECTION =====
```

```
Average: 95 -> Honor Roll: True
```

```
Average: 90 -> Honor Roll: False
```

```
Average: 89 -> Honor Roll: False
```

```
===== 8. REMOVE A GRADE =====
```

```

===== 7. HONOR ROLL DETECTION =====
Average: 95 -> Honor Roll: True
Average: 90 -> Honor Roll: False
Average: 89 -> Honor Roll: False

===== 8. REMOVE A GRADE =====
Initial grades: [95.0, 80.0, 70.0]
After removing index 1: [95.0, 70.0]
After removing value 95.0: [70.0]
Invalid index: Grade index is out of range.
Invalid value: Grade 100.0 does not exist.

===== 9. STUDENT SUMMARY REPORT =====
===== STUDENT REPORT =====
Student ID: 001
Student Name: John Doe
Number of Grades: 3
Average Grade: 85.17
Letter Grade: B
Pass/Fail: Passed
Honor Roll: False
○ (.venv) PS C:\Programming\Educacion\taller_coding_standards>

```

Step 5 – Final report

Initial Report:

***** Module test

test.py:1:0: C0114: Missing module docstring (missing-module-docstring)

test.py:2:0: C0115: Missing class docstring (missing-class-docstring)

test.py:2:0: C0103: Class name "student" doesn't conform to PascalCase naming style (invalid-name)

test.py:7:8: C0103: Attribute name "isPassed" doesn't conform to snake_case naming style (invalid-name)

test.py:3:4: E0213: Method '__init__' should have "self" as first argument (no-self-argument)

test.py:3:20: W0622: Redefining built-in 'id' (redefined-builtin)

test.py:10:4: C0116: Missing function or method docstring (missing-function-docstring)

test.py:10:4: C0103: Method name "addGrades" doesn't conform to snake_case naming style (invalid-name)

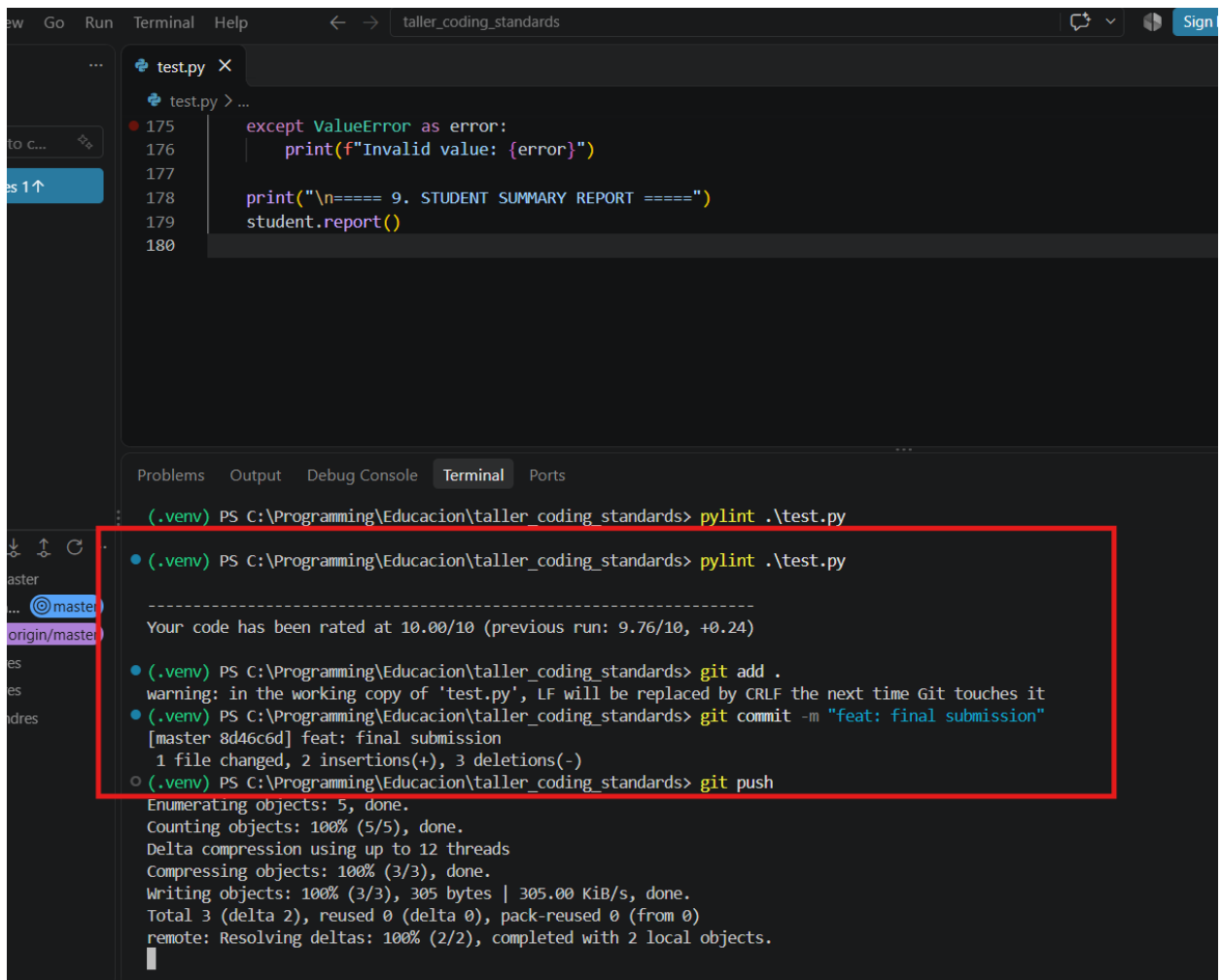
test.py:13:4: C0116: Missing function or method docstring (missing-function-docstring)

test.py:17:8: W0612: Unused variable 'avg' (unused-variable)
test.py:19:4: C0116: Missing function or method docstring (missing-function-docstring)
test.py:19:4: C0103: Method name "checkHonor" doesn't conform to snake_case naming style (invalid-name)
test.py:20:11: E1101: Instance of 'student' has no 'calcAverage' member; maybe 'calcaverage'? (no-member)
test.py:23:4: C0116: Missing function or method docstring (missing-function-docstring)
test.py:23:4: C0103: Method name "deleteGrade" doesn't conform to snake_case naming style (invalid-name)
test.py:26:4: C0116: Missing function or method docstring (missing-function-docstring)
test.py:30:33: E1101: Instance of 'student' has no 'letter' member (no-member)
test.py:33:0: C0116: Missing function or method docstring (missing-function-docstring)

Your code has been rated at 0.91/10 (previous run: 0.91/10, +0.00)

Final Report:

Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)



```
test.py
175 except ValueError as error:
176     print(f"Invalid value: {error}")
177
178 print("\n===== 9. STUDENT SUMMARY REPORT =====")
179 student.report()
180

(.venv) PS C:\Programming\Educacion\taller_coding_standards> pylint .\test.py

(.venv) PS C:\Programming\Educacion\taller_coding_standards> pylint .\test.py
-----
Your code has been rated at 10.00/10 (previous run: 9.76/10, +0.24)

(.venv) PS C:\Programming\Educacion\taller_coding_standards> git add .
warning: in the working copy of 'test.py', LF will be replaced by CRLF the next time Git touches it
(.venv) PS C:\Programming\Educacion\taller_coding_standards> git commit -m "feat: final submission"
[master 8d46c6d] feat: final submission
1 file changed, 2 insertions(+), 3 deletions(-)
(.venv) PS C:\Programming\Educacion\taller_coding_standards> git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 305 bytes | 305.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
```

Step 6 – Repository URL

https://github.com/porrasmdz/coding_standards_workshop

Conclusions

In this practice we demonstrated the use of a linting tool in order to enforce good practices in an educational test python program to calculate students' grade. This practice was important because we could install, lint and fix the code iteratively reflecting a real world example of writing code following good practices.

Recommendations

It could also be useful to use this kind of tools in group works in order to keep consistency through a bigger project.

It would also be interesting to test a code coverage tool and build a workflow that allows or not code submissions if they follow a threshold in coverage percentage and linting grade.